

```
main executable and any
library
```

5.2.2 Predicate

The predicate is any D expression. You can construct predicates using the many built-in variables or arguments provided by the probe. The following table shows a few examples. The action is executed only when the predicate evaluates to true.

Predicate	Explanation
<code>cpu == 0</code>	True if the probe executes on <code>cpu0</code>
<code>pid == 1029</code>	True if the pid of the process that caused the probe to fire is 1029
<code>uid == 123</code>	True if the probe is fired by a process owned by userid 123
<code>execname == "mysql"</code>	True if the probe is fired by the <code>mysql</code> process
<code>execname != "sched"</code>	True if the process is not the scheduler (<code>sched</code>)
<code>ppid !=0 && arg0 == 0</code>	True if the parent process id is not 0 and the first argument is 0

5.2.3 Action

The action section is a series of action commands separated by semicolon characters (;). The following table shows a few examples. The action is executed only when the predicate evaluates to true.

Action	Explanation
<code>printf()</code>	Print something using C-style <code>printf()</code> command
<code>ustack()</code>	Print the user level stack
<code>trace()</code>	Print the given variable

5.2.4 Developing a D Script

The following `dtrace` command prints all the functions that process id 1234 calls. The `-n` option specifies a probe name to trace.

```
# dtrace -n pid1234:::entry
```

The following example shows how to use the above